

BuildTrack — Architecture, Workflows & Repo Starter

This new canvas focuses on **clear architecture diagrams, workflow diagrams, and a ready-to-copy repo layout with placeholder files** so your team can start implementing immediately. I created concise **Mermaid diagrams** for architecture and critical flows (booking/payment/notifications) and a full repo scaffold (backend + frontend + infra) with suggested placeholder files and short contents for each.

Tip: the mermaid blocks below can be rendered by tools that support mermaid. If your viewer doesn't render them, they still document the flow clearly for developers.

1. High-level Architecture (Mermaid)

```
graph LR
    subgraph Clients
        P[Parent Web/Mobile]
        C[Child App]
        B[Builder Web/Mobile]
        Admin[Admin Web]
    end

    CDN[CDN / Static Assets]
    APIGW[API Gateway / Load Balancer]
    REST[REST / GraphQL API Servers]
    SOCKET[Socket.IO / WebSocket Cluster]
    WORKER[Worker Fleet (Celery/RQ)]
    POSTGRES[(Postgres DB)]
    REDIS[(Redis Cache & Locks)]
    S3[(S3 / Object Storage)]
    PAY[Payment Gateway]
    MSG[Message Broker (RabbitMQ/Kafka)]
    MON[Monitoring / Logs]

    Clients --> CDN
    Clients --> APIGW
    APIGW --> REST
    APIGW --> SOCKET
    REST --> POSTGRES
    REST --> REDIS
    REST --> S3
    REST --> PAY
    REST --> MSG
    MSG --> WORKER
```

```
WORKER --> S3
WORKER --> POSTGRES
SOCKET --> POSTGRES
SOCKET --> REDIS
REST --> MON
WORKER --> MON

style CLIENTS fill:#f9f,stroke:#333,stroke-width:1px
```

Notes:

- API servers handle REST/GraphQL requests. Socket cluster (Socket.IO or equivalent) handles real-time push.
- Redis used for caches and distributed locks (prevent duplicate payments). Message broker for reliable async tasks.
- Workers process thumbnails, expiries, reminders and heavy jobs.
- S3 stores media and thumbnails. Postgres is the system of record.

2. Booking → Payment → Start Sequence (Mermaid Sequence)

```
sequenceDiagram
    participant Child
    participant Parent
    participant API
    participant DB
    participant PaymentGW
    participant Socket

    Child->>API: POST /children/{child}/services/{service}/request
    API->>DB: create subscription(status=pending)
    API->>Socket: emit(child_requested -> family_room)
    Socket-->>Parent: real-time notification

    Parent->>API: POST /parents/{id}/subscriptions/{sub}/approve
    API->>DB: BEGIN TRANSACTION; SELECT FOR UPDATE subscription
    alt subscription still pending
        API->>PaymentGW: create charge (idempotency key)
        PaymentGW-->>API: success
        API->>DB: update subscription (status=active,payment_status=paid)
        API->>Socket: emit(subscription_approved -> family_room; notify builder)
        API-->>Parent: 200 OK (approved)
    else already approved
        API-->>Parent: 409 Conflict (already approved)
```

```
end
DB->>Socket: (persist notification row)
```

Key details:

- `SELECT ... FOR UPDATE` or DB row lock prevents race conditions when multiple parents try to approve simultaneously.
- Payment gateway calls should use **idempotency keys** and be retried safely.
- All notifications saved to `notifications` table before emitting.

3. Notification Flow (Mermaid Flowchart)

```
flowchart TD
    A[Event: builder updates progress] --> B[API: create notification row in Postgres]
    B --> C[SocketIO.emit to family_room & builder_room]
    C --> D[Client(s) receive notification in real-time]
    B --> E[Notification Center (history) available via GET /families/{id}/notifications]
```

Notes: Save-then-emit ensures reliable history and that offline users can fetch missed notifications.

4. Media Upload & Thumbnail Workflow

```
flowchart LR
    U[Frontend] -->|request presign| API
    API --> S3_presign[Generate presigned URL]
    S3_presign --> U
    U -->|upload directly| S3
    S3 --> Worker[Thumbnail Worker]
    Worker --> S3_thumb[store thumbnail]
    Worker --> Postgres[update media record]
    Postgres --> API[media list served]
```

Limits: videos <= 50MB; validate at front-end and presign policy.

5. Expiry Background Job (pseudo)

- Worker cron (every 30 min):
- `SELECT id FROM subscriptions WHERE status='booked' AND expires_at < now()`

- For each id: update `status='expired'`, push notifications to family & builder, free slots.

6. Repo Starter — Folder Structure & Placeholder Files

Below is a ready-to-create repo structure. I included minimal placeholder contents for critical files so you can `git init` and start coding.

6.1 Root

```
buildtrack/  
├─ backend/  
├─ frontend/  
├─ infra/  
├─ docker-compose.yml  
└─ README.md
```

6.2 Backend (FastAPI + SQLAlchemy + Socket.IO)

```
backend/  
├─ app/  
│   ├── main.py  
│   ├── config.py  
│   ├── db.py  
│   └── models/  
│       ├── __init__.py  
│       ├── users.py  
│       ├── families.py  
│       ├── builders.py  
│       └── reviews.py  
│   ├── routes/  
│       ├── auth.py  
│       ├── families.py  
│       ├── builders.py  
│       ├── subscriptions.py  
│       ├── payments.py  
│       └── notifications.py  
│   ├── services/  
│       ├── presign.py  
│       ├── thumbnailer.py  
│       └── payment_gateway.py  
│   └── socketio_server.py  
└─ workers/  
    └── expiry_checker.py
```

```
|   └─ thumbnail_worker.py
|   └─ alembic/
|   └─ requirements.txt
|   └─ Dockerfile
|   └─ .env.example
```

Placeholder:

```
from fastapi import FastAPI
from .socketio_server import socketio

app = FastAPI()

@app.get('/health')
def health():
    return {'status': 'ok'}

# mount other routers here

if __name__ == '__main__':
    # run with `uvicorn --reload app.main:app`
    import uvicorn
    uvicorn.run('app.main:app', host='0.0.0.0', port=8000, reload=True)
```

Placeholder:

```
from fastapi import FastAPI
from fastapi_socketio import SocketManager

# Note: use a proper socket.io setup in production (engineio + redis adapter)
socket_manager = SocketManager()

def init_socket(app: FastAPI):
    socket_manager.attach(app)

# in main.py call: init_socket(app)
```

Placeholder:

```
from fastapi import APIRouter, Depends, HTTPException
from app.db import get_db, SessionLocal

router = APIRouter()
```

```
@router.post('/parents/{parent_id}/subscriptions/{subscription_id}/approve')
def approve(parent_id: int, subscription_id: int):
    # TODO: implement SELECT FOR UPDATE transaction logic here
    return { 'message': 'stub' }
```

.env.example

```
DATABASE_URL=postgresql://user:pass@db:5432/buildtrack
REDIS_URL=redis://redis:6379/0
S3_BUCKET=my-bucket
AWS_REGION=us-east-1
PAYMENT_API_KEY=sk_test_xxx
SECRET_KEY=supersecret
```

6.3 Frontend (Next.js + Tailwind)

```
frontend/
├─ public/
├─ src/
│  ├─ pages/
│  │  ├─ index.tsx
│  │  ├─ dashboard.tsx
│  │  ├─ builders/[id].tsx
│  │  └─ subscriptions/[id].tsx
│  ├─ components/
│  │  ├─ NavBar.tsx
│  │  ├─ Notifications.tsx
│  │  ├─ BuilderCard.tsx
│  │  └─ reviews/
│  │     ├─ ReviewList.tsx
│  │     └─ ReviewForm.tsx
│  ├─ hooks/
│  │  ├─ useSocket.ts
│  │  └─ useAuth.ts
│  └─ lib/
│     └─ api.ts
├─ package.json
└─ tailwind.config.js
```

****Placeholder: **`**

```
import { useEffect } from 'react'
import { io } from 'socket.io-client'

export default function useSocket(familyId: number) {
  useEffect(() => {
    const socket = io(process.env.NEXT_PUBLIC_SOCKET_URL as string)
    socket.emit('join_family', { family_id: familyId })
    socket.on('notification', (data) => {
      console.log('notif', data)
    })
    return () => { socket.disconnect() }
  }, [familyId])
}
```

6.4 Infra (docker-compose)

*** (placeholder)**

```
version: '3.8'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: buildtrack
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - db_data:/var/lib/postgresql/data
  redis:
    image: redis:7
  backend:
    build: ./backend
    env_file: ./backend/.env.example
    depends_on: [db, redis]
  frontend:
    build: ./frontend
    env_file: ./frontend/.env.local
    depends_on: [backend]
volumes:
  db_data:
```

7. Starter Code Snippets (critical flows)

7.1 Transaction-safe Approve (psuedocode, Python + psycopg2)

```
conn = psycopg2.connect(DATABASE_URL)
with conn:
    with conn.cursor() as cur:
        cur.execute("BEGIN")
        cur.execute("SELECT status, payment_status FROM subscriptions WHERE
id=%s FOR UPDATE", (sub_id,))
        row = cur.fetchone()
        if not row: raise Exception('not found')
        if row[0] != 'pending': raise Exception('already processed')
        # call payment gateway with idempotency key
        # on success:
        cur.execute("UPDATE subscriptions SET status='active',
payment_status='paid', start_date=NOW() WHERE id=%s", (sub_id,))
        conn.commit()
```

7.2 Presign S3 (Flask example)

```
import boto3
s3 = boto3.client('s3')

def presign_upload(key, content_type, bucket):
    return s3.generate_presigned_post(Bucket=bucket, Key=key, Fields={'Content-
Type':content_type}, ExpiresIn=3600)
```

7.3 Notify & Save (Socket + DB)

```
def notify_and_save(family_id, payload):
    # save to DB
    cur.execute("INSERT INTO notifications (family_id, type, message, data)
VALUES (%s,%s,%s,%s)", (family_id, payload['type'], payload.get('message'),
json.dumps(payload)))
    conn.commit()
    # emit
    socketio.emit('notification', payload, room=f'family_{family_id}')
```